0ICPC455

AY 2024-25

Machine Learning Lab

Quality Laboratory Manual

Experiment No. 6

Study and Implementation of Random Forest Algorithm

Course Instructor -

DR. TAHSEEN A. MULLA

ASSOCIATE PROFESSOR

Experiment No. 6

Title of Experiment: To study and implement Random Forest Algorithm

Aim of Experiment: To implement and understand the working of Random Forest algorithm, an learning method used for both classification and regression tasks in Machine Learning

System Requirements – Win 8 and above OS, 4GB RAM, 2.33 GHz Processor

Software/s Needed for Experiment – Jupyter Notebook/ Anaconda Navigator/ Google Colaboratory/ Spyder, Python 3.x [With libraries such as Numpy, Pandas, Matplotlib and Scikit-Learn]

Experiment Outcomes –

1. Understand the concept and principles of Random Forest algorithm
2. To understand and implement the Random Forest algorithm including ensemble learning, bootstrapping and aggregation
3. Able to evaluate the performance of Random Forest using appropriate metrics
4. To train and use a Random Forest model for classification tasks on real-world datasets

Theory –

Random Forest is an ensemble learning method that combines multiple decision trees to create a "forest" for better performance and generalization. Each tree in the forest is trained on a different subset of the data using a technique called bagging (Bootstrap Aggregating), which reduces overfitting compared to a single decision tree.

Random Forest works by aggregating the outputs of several decision trees:

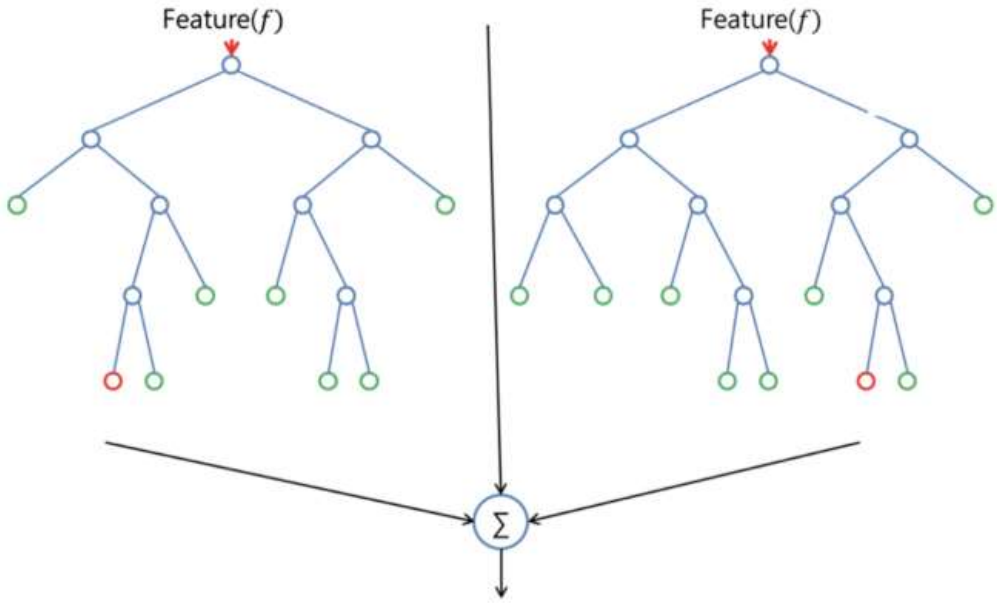
- **For Classification**: Each tree votes for a class, and the class with the most votes is the final output.
- **For Regression**: The output is the average of the predictions from all trees

Random forest is a flexible, easy-to-use machine learning algorithm that produces even without hyper-parameters tuning a great result most of the time. It is also one of the most used algorithms due to its simplicity and diversity

How Random Forest works?

One big advantage of random forest is that it can be used for both classification and regression problems, which form the majority of current machine learning systems.

Let's look at random forest in classification, since classification is sometimes considered the building block of machine learning. Below you can see how a random forest model would look like with two trees:



Random Forest in Classification and Regression –

Random forest has nearly the same hyperparameters as a decision tree or a bagging classifier. Fortunately, there's no need to combine a decision tree with a bagging classifier because you can easily use the classifier-class of random forest. With random forest, you can also deal with regression tasks by using the algorithm's regressor.

Random forest adds additional randomness to the model, while growing the trees. Instead of searching for the most important feature while splitting a node, it searches for the best feature among a random subset of features. This results in a wide diversity that generally results in a better model.

Therefore, in a random forest classifier, only a random subset of the features is taken into consideration by the algorithm for splitting a node. You can even make trees more random by additionally using random thresholds for each feature rather than searching for the best possible thresholds (like a normal decision tree does)

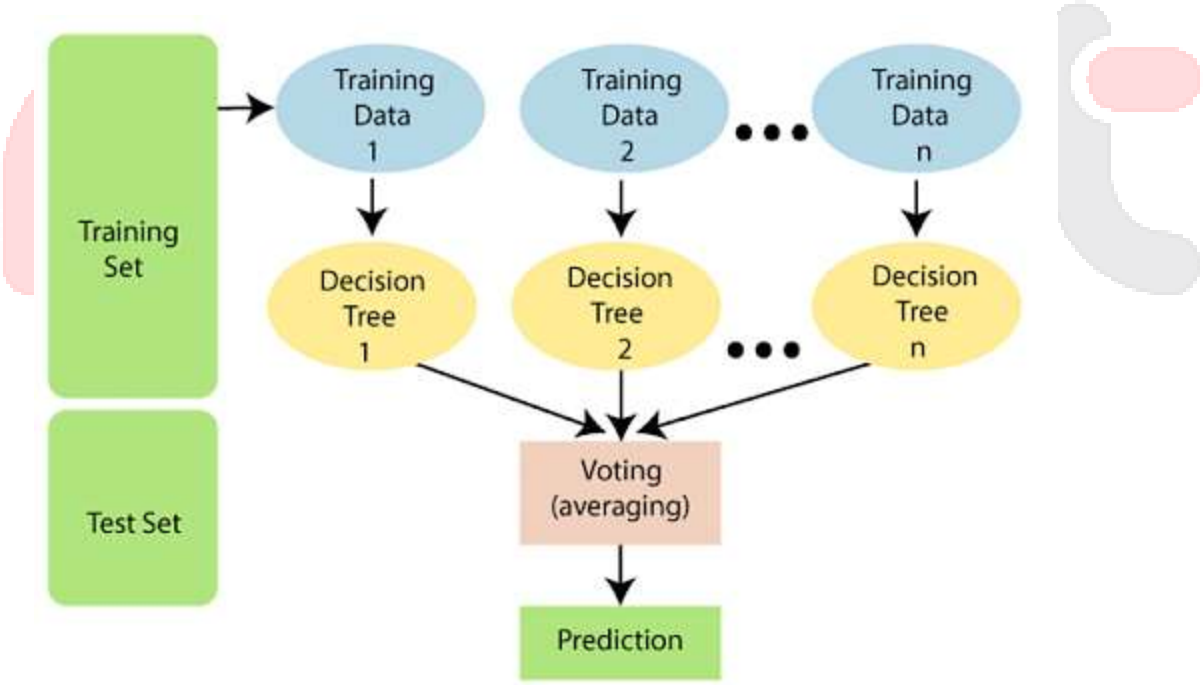
Random Forest Models v/s Decision Trees –

While a random forest model is a collection of decision trees, there are some differences.

If you input a training dataset with features and labels into a decision tree, it will formulate some set of rules, which will be used to make the predictions.

For example, to predict whether a person will click on an online advertisement, you might collect the ads the person clicked on in the past and some features that describe their decision. If you put the features and labels into a decision tree, it will generate some rules that help predict whether the advertisement will be clicked or not. In comparison, the random forest algorithm randomly selects observations and features to build several decision trees and then averages the results.

Another difference is "deep" decision trees might suffer from overfitting. Most of the time, random forest prevents this by creating random subsets of the features and building smaller trees using those subsets. Afterwards, it combines the subtrees. It's important to note this doesn't work every time and it also makes the computation slower, depending on how many trees the random forest builds.



Procedure to implement Random Forest algorithm:

1. **Data Collection:** Obtain or create a dataset suitable for classification or regression. A dataset such as Iris, which classifies iris flowers into three species – Setosa, Versicolour and Virginica

2. **Data Preprocessing:**
   a. Load the dataset into a Pandas DataFrame.
   b. Handle missing values, outliers, and encode categorical variables if necessary.

3. **Exploratory Data Analysis (EDA):** Visualize relationships between the features and the target class using scatter plots, pair plots and correlation matrices

4. **Splitting the Data:** Split the dataset into training and test sets to evaluate the model's performance.

5. **Implementing Decision Tree Algorithm:**
   a. Use the Scikit-learn library [RandomForestClassifier] to implement Random Forest model.
   b. Train the model using the training data.

6. **Model Prediction:**
   a. Use the trained model to make predictions on the test data.
   b. Generate a confusion matrix and classification report to assess accuracy

7. **Model Evaluation:**
   a. Evaluate the model using metrics such as accuracy, precision, recall, F1-score
   b. Determine the feature importance and visualize it to understand the contribution of each feature

Sample Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.datasets import load_iris

# Step 1: Data Collection (Iris dataset)
iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = pd.DataFrame(iris.target, columns=['species'])

# Step 2: Data Preprocessing
```

```
# The Iris dataset is already clean with no missing values or
outliers.

# Step 3: Exploratory Data Analysis (EDA)
sns.pairplot(pd.concat([X, y], axis=1), hue='species',
palette='Set1')
plt.show()

# Step 4: Splitting the Data
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Step 5: Implementing the Random Forest Algorithm
rf = RandomForestClassifier(n_estimators=100, criterion='entropy',
random_state=42)
rf.fit(X_train, y_train.values.ravel())

# Step 6: Model Prediction
y_pred = rf.predict(X_test)

# Comparing Actual vs Predicted
comparison_df = pd.DataFrame({'Actual': y_test.values.ravel(),
'Predicted': y_pred})
print(comparison_df)

# Step 7: Model Evaluation
# Confusion Matrix
conf_matrix = confusion_matrix(y_test, y_pred)
print(f"Confusion Matrix:\n{conf_matrix}")

# Classification Report
class_report = classification_report(y_test, y_pred,
target_names=iris.target_names)
print(f"Classification Report:\n{class_report}")

# Feature Importance
feature_importances = pd.Series(rf.feature_importances_,
index=iris.feature_names)
feature_importances.sort_values().plot(kind='barh', color='skyblue')
plt.title('Feature Importance')
plt.xlabel('Importance Score')
plt.ylabel('Features')
plt.show()
```

Observations -

- Record the predicted classes for the test data
- Observe the performance of the model using the confusion matrix and classification report
- Analyze which feature have the highest importance scores

Conclusion –

Hence, the model summarizes the findings from the experiment, such as effectiveness of the Decision Tree model in classifying iris species

References –

a. Textbook –

i. Machine Learning with Python – An approach to Applied ML – Abhishek Vijayvargiya, BPB Publications, 1st Edition 2018
ii. Machine Learning, Tom Mitchell, McGraw Hill Education, 1st Edition 1997

b. Online references –

i. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html
ii. https://www.kaggle.com/code/dansbecker/randomforests
iii. https://h2o.ai/wiki/random-forest/

Expected Oral Questions –

1. What is a Random Forest and how does it work?
2. How does a Random Forest differ from a single decision tree?
3. What is the purpose of using multiple trees in a Random Forest?
4. What types of problems are Random Forest typically used for?
5. What is the significance of using random subsets of features for each split in a Random Forest?
6. How is the final output of a Random Forest determined?
7. How do you evaluate the feature importance in a Random Forest algorithm?
8. What are the advantages of using a Random Forest compared to a single Decision Tree?
9. How do you interpret the output of a Random Forest for a classification problem?
10. In what scenarios would you prefer a Random Forest over the other Machine Learning algorithms?

FAQ's in Interview –

1. What do you mean by Random Forest algorithm?
2. Why is Random Forest Algorithm so popular?
3. What are the main advantages of using a Random Forest?
4. How do you handle missing values in a Random Forest model?
5. What are the key hyper-parameters of a Random Forest and how do they affect the model?